

TECHNICAL DEEP DIVE

从压缩注意力到可执行推理：DeepSeek V4 如何落入 vLLM

基于技术演讲与配套材料整理

配套材料：[vLLM-DSV4-Internals-0508-share.pdf](#)

2026-09-05

沿注意力设计、异构缓存、内存布局与双流执行的因果链，理解百万 token 推理背后的系统工程。

原视频：[大家一起来学 DeepSeek-v4](#) · 配套资料：[vLLM DSV4 Internals](#)

长上下文首先是模型问题，但不会止于模型。

当历史增长到百万 token 量级，标准 KV Cache 的容量、访存量和注意力计算都会持续增加。DeepSeek V4 通过压缩历史、稀疏选择和局部窗口降低压力，却也把原本相对同构的缓存拆成多类状态：它们具有不同的压缩率、生命周期、物理尺寸和增长规律。

因此，vLLM 面对的任务不只是实现新的注意力算子。系统还要回答：压缩状态如何分配？逻辑 token 怎样映射到物理条目？长请求与大量短请求如何共享显存？Indexer 和主注意力怎样并行？哪些小算子值得融合？

本文依据演讲 PPT 与转写材料，沿“模型结构 → 缓存容量 → 内存布局 → Decode 执行 → 源码入口”的链路展开。页面可以确认的内容按 PPT 页码标注；仅由口述支持的重要数字、性能观察和后续规划，通过脚注注明。材料未实现的实现细节、精度消融和性能条件统一列为待核验项。

适读人群、前置知识与阅读目标

本文适合了解 Transformer、注意力和 GPU 推理基础，但尚不熟悉 DeepSeek V4 或 vLLM 混合缓存实现的工程师。

阅读前最好具备以下基础：

- 理解自回归推理中的 Prefill、Decode 与 KV Cache。
- 了解 Multi-Query Attention、MoE、RoPE、FP16、FP8 和 GPU kernel launch。
- 能区分逻辑 token、压缩条目、逻辑 block、物理 block 和分配对齐单位。
- 知道滑动窗口缓存与随完整序列增长的缓存具有不同生命周期。

读完后，你应该能够：

- 解释 CSA 与 HCA 如何表示长期历史，以及 SWA 为什么仍不可缺少。
- 根据 `block_size`、`storage_block_size`、表示维度和数据类型复核缓存开销。
- 理解异构状态为什么需要 Hybrid KV Cache Manager。
- 找到 C4A Decode 中默认流、Indexer 流和 Flash MLA 的同步点。
- 区分页面事实、口述观察、合理推断与仍需源码验证的结论。

一、模型压缩历史，系统却得到更多状态

DeepSeek V4 面向约一百万 token 上下文时，不能只是扩大标准注意力的 KV Cache。序列越长，缓存容量、访存量和计算量都会持续增加；即使显存勉强容纳，推理系统也未必能以可接受的成本调度这些数据。

模型侧选择压缩与稀疏化，新的系统矛盾随之出现：保存的历史条目减少了，缓存类型却增加了。

为什么先看整体架构图？因为后文涉及的压缩历史、Indexer Cache、局部窗口、compressor state 和 MoE 执行路径，都能在这张图中找到模型侧源头。

vLLM

Recap on Model Architecture

1. mHC
2. Compressed sparse attention
3. Highly compressed attention
4. MoE
 - a. Hash routing
 - b. MegaMoE

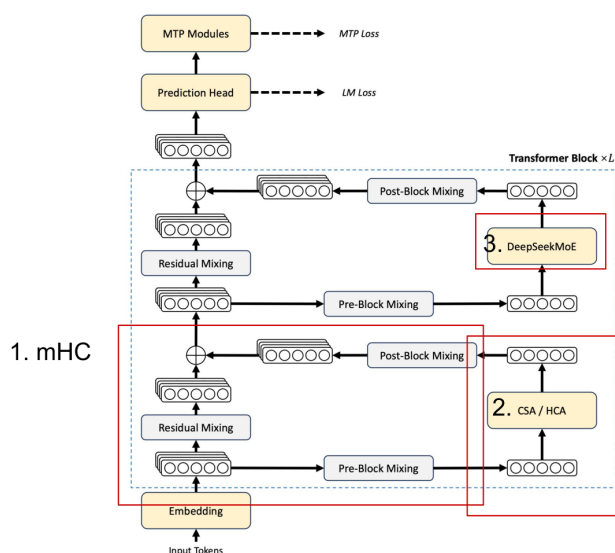


图 1：DeepSeek V4 模型结构总览。来源：演讲 PPT，第 2 页。

数据从 **Input Tokens** 经 **Embedding** 进入重复的 Transformer Block。图中的“ $\times L$ ”只表示块会重复，不能据此确定层数；材料也没有给出此处的张量形状、专家数量或训练损失权重。

每个块包含三组需要区分的设计。

第一组是 **mHC**。材料将其描述为一种多路残差增强结构，覆盖块内的预混合、残差混合和后混合路径。进入注意力或 MoE 前，多路表示会恢复成下游模块所需的单路 hidden state。演讲者所说的“对后续接口没有影响”，只表示下游模块仍接收单路表示，不能扩大为没有额外计算、数值变化或实现成本。[^T02T03]

第二组是注意力分支采用的两类层配置：

- **CSA (Compressed Sparse Attention, 压缩稀疏注意力, 也称 C4A)**：以约 4:1 的比例压缩历史，再通过 Indexer 选出 Top-k 条目参与当前注意力。
- **HCA (Highly Compressed Attention, 高压压缩注意力, 也称 C128A)**：以约 128:1 的比例压缩历史；其架构图不再展示 CSA 式的显式 Top-k 选择路径。

CSA 与 HCA 不是同一层内同时执行的两条注意力路径，而是 Transformer Block 注意力分支采用的两类层配置。在页面所示模型统计中，C4A 层与 C128A 层分层交错；两者的压缩率、缓存数量和访问方式均不同。

第三组是 DeepSeekMoE。图中包含 Hash Routing 和 MegaMoE。演讲者称，前三层通过固定 hash table 为 token 指定专家，而不是由 gate 动态选择，并转述相关报告认为这种方式可能更易训练。材料没有训练对照数据，因此这只能视为带归属的设计说明，不能作为实验结论。[^T03]

MegaMoE

指向执行侧：把专家计算、通信和部分零碎操作整合进较大的执行单元，以减少碎片化调度与 kernel launch。其证据边界将在第六节讨论。

图顶部还有 Prediction Head 与 MTP Modules，分别连接 LM Loss 和 MTP Loss。箭头只表达图中的训练目标关系；材料没有说明损失权重，也没有说明推理阶段是否启用 MTP，不能由此推导在线执行行为。

把模型结构映射到系统职责，可以得到下面的问题地图：

模型结构	新增或变化的状态	vLLM 的系统职责
mHC 多路残差流	块内多路表示及混合过程	保持下游接口并安排表示转换
CSA 层	压缩历史、Indexer 信息和局部未压缩状态	协调缓存分配、Top-k 选择与主注意力
HCA 层	压缩率更高、增长更慢的历史条目	按不同于原生 token 的尺度管理存储
DeepSeekMoE	路由结果、专家计算和通信任务	调度专家路径并减少零碎启动

若历史长度为 N ，暂时忽略边界和近期窗口，CSA 的压缩条目量级约为 $N/4$ ，HCA 约为 $N/128$ 。例如 $N=128$ 时，两者大致对应 32 条和 1 条压缩历史。

这只是数量关系，不是完整的显存账目。真实状态还包括 Indexer Cache、主注意力 KV、压缩器状态，以及保存近期 token 的 SWA (Sliding Window Attention，滑动窗口注意力)。

压缩注意力没有消除缓存管理，而是把一种随 token 增长的同构 KV Cache 转换为多种尺度、用途和生命周期不同的状态。下面先从结构更复杂的 CSA 看这些状态为何出现。

二、CSA：容量压缩之后，为什么还要稀疏选择

CSA 同时面对三个目标：

- 近期 token 尽量保持原始粒度。
- 长期历史不能继续按原生 token 数量增长。
- 当前 query 不应遍历全部压缩历史执行主注意力。

因此，CSA 使用两级缩减：

- 1 Token compression 减少长期历史实际保存的条目数。

2 Top-k selection 限制当前 Decode step 真正访问的历史范围。

前者控制容量，后者控制本次计算量，两者不能互相替代。

为什么看下面这张图？它把 CSA 拆成两条数据路径：一条生产主注意力需要的 KV，另一条负责给压缩历史打分。只有沿箭头追踪两条路径，才能看清“保存更少”和“读取更少”的差别。

vLLM

Recap on Model Architecture: CSA

https://vllm.ai/blog-assets/interactive_pages/c4a.html

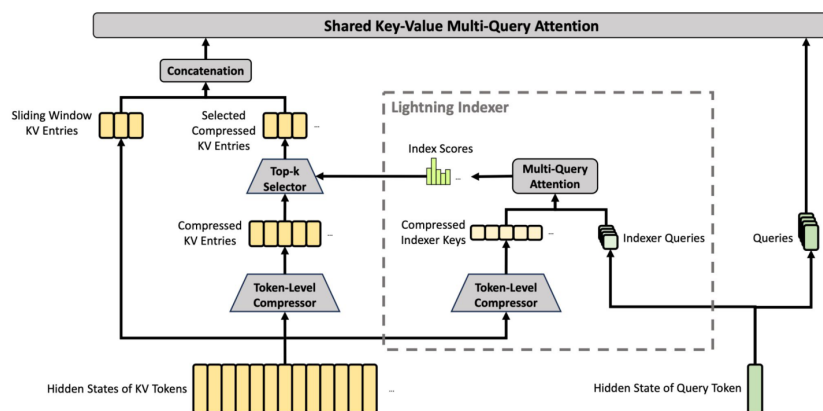


图 2：CSA 的主 KV 路径与 Lightning Indexer 路径。来源：演讲 PPT，第 3 页。

主 KV 路径：长期压缩，近期直达

图左下角的 KV token hidden states 向上分成两路：

- 一路直接形成 **Sliding Window KV Entries**，保存查询附近的近期 token。
- 另一路经过 **Token-Level Compressor**，形成 **Compressed KV Entries**，用较少条目表示更早的历史。

演讲示例中的主压缩比例约为 4:1，即每 4 个原生 token 对应一个压缩条目。相邻更新使用覆盖 8 个 token 的压缩器状态，并以 4 个新 token 为推进步长。

条目数量降到四分之一后，压缩历史仍会随序列增长。若当前 query 每次都与全部压缩条目计算注意力，计算规模只会增长得更慢，并不会成为固定值。

Indexer 路径：决定当前读取哪些历史

Lightning Indexer 是一条轻量索引路径：

- 历史 hidden states 经压缩得到 **Compressed Indexer Keys**。
- 当前 query hidden state 生成 **Indexer Queries**。
- 两者进入图中的 Indexer Multi-Query Attention，得到候选条目的 **Index Scores**。

4 Top-k Selector 按分数选择本次访问的压缩历史。

选中的 **Selected Compressed KV Entries** 随后与 **Sliding Window KV Entries** 拼接，再进入 Shared Key-Value Multi-Query Attention 所对应的主注意力路径。

最终输入可以写成：

$$KV_actual = KV_selected-history \mathbin{\|} KV_recent-window$$

其中， $\mathbin{\|}$ 表示拼接。

因此，Indexer 决定访问位置，主注意力读取对应 KV 并计算输出；长期历史按相关性取回，近期 token 则由局部窗口直接保留。

相邻两次压缩更新如何衔接

材料确认了三个量：

- 压缩比例约为 4:1。
- compressor state 的示例覆盖范围为 8 个 token。
- 相邻更新之间新增 4 个 token，即推进步长为 4。

材料没有确认 warm-up、padding、首个压缩条目的发射时刻，也没有给出边界对齐规则。因此，下面只表示进入稳定更新阶段后的相邻两次更新，不绑定绝对 token 编号。

相对阶段	本轮新增输入	Compressor state	可以确认的结果
更新 u	新增 4 个 token	使用当前覆盖范围内的状态	形成或更新一个压缩条目
两次更新之间	尚未积累下一组 4 个新 token	保留既有状态并吸收新输入	新 token 继续由 Sliding Window KV 保持可见
更新 u+1	再新增 4 个 token	状态窗口向前推进 4 个位置，保留重叠信息	形成下一个压缩条目

如果用长度为 8 的抽象窗口表示稳定阶段，可写成：

更新 u: [a b c d e f g h]
更新 u+1: [e f g h i j k l]

相邻更新使用重叠的状态窗口。两个窗口重叠 4 个 token，但这种重叠如何影响单个压缩条目的语义覆盖范围，材料无法确认。

同样需要区分两种“窗口”：

- 8-token 是演讲示例中的 C4 compressor state 覆盖范围。
- Sliding Window KV 保存近期原始 KV，但其精确窗口长度没有在材料中确认。

真实实现从何时发射首个条目、warm-up padding，以及序列边界如何对齐，都需要结合对应版本源码核验。阶段是否

这正是 SWA 必须存在的原因：压缩条目按步长更新，新 token 却必须立即对查询可见。尚未进入下一次压缩更新的近期 token，不会从上下文中消失，而是继续保存在 Sliding Window KV 中。

三种机制的职责边界

机制	改变的对象	不改变的对象	主要作用
Token compression	长期历史实际保存的条目数	当前 query 是否访问全部压缩条目	降低缓存容量增长速度
Top-k selection	当前计算访问的历史范围	已保存的压缩条目总数	限制单次注意力计算量
Sliding Window KV	近期 token 的原始 KV 覆盖	长期历史的压缩方式	补足尚未压缩的局部信息

若原始历史包含 N 个 token，按 4:1 粗略换算后，长期条目数量约为 N/4。Indexer 再从中选择 k 个参与当前计算。

演讲中出现了“最多 512 个”的说法，但 PPT 只标出 Top-k，没有确认 512 是固定模型常量、默认配置还是特定示例，因此不能把它写成所有 CSA 配置通用的参数。[^T04]

材料能够确认职责与数据流，但尚不能确认：

- Token-Level Compressor 的具体算法和压缩维度。
- 两条 compressor 路径是否共享参数。
- Sliding Window KV 的精确长度。
- Top-k 的固定取值及配置方式。
- 首个压缩条目的发射时刻和边界对齐规则。
- 单个压缩条目的严格语义覆盖区间。

演讲者还区分了容量压缩与计算稀疏：稀疏选择本身不会减少已经保存的历史缓存，KV Cache 容量下降主要来自 token compression。[^T06]

CSA 用较温和的压缩保留更多历史条目，因此仍需 Indexer 控制实际访问范围。HCA 则把压缩率提高到另一个数量级，由此缩短架构图中的历史访问路径。

三、HCA：更少的历史条目，更直接的访问路径

HCA 与 CSA 面对相同的问题：既要保存长期历史，也要让最近到达的 token 立即可见。差别在于，HCA 采用约 128:1 的压缩率，也就是约 128 个原生 token 对应一个历史条目。

为什么还要单独看 HCA 图？因为更高的压缩率不仅改变容量，也改变架构图中的访问路径：压缩历史、局部窗口和当前 query 直接进入同一个主注意力模块，不再展示 CSA 的显式索引分支。

Recap on Model Architecture: HCA

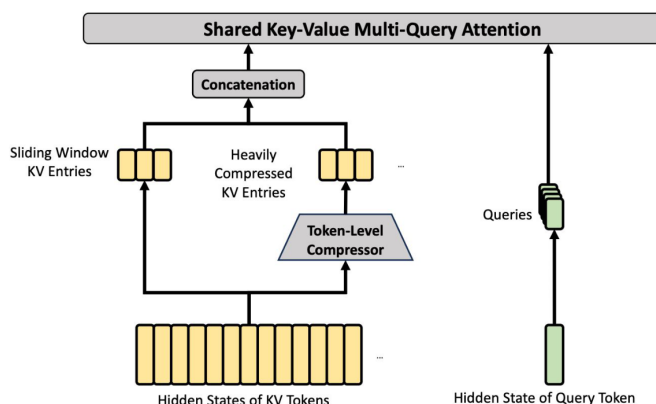


图 3：HCA 的架构级数据流。来源：演讲 PPT，第 4 页。

图中有四类关键元素：

- 1 **Heavily Compressed KV Entries**: 历史 token 经 Token-Level Compressor 形成的高压缩 KV。
- 2 **Sliding Window KV Entries**: 保存近期、尚未进入压缩历史的 token。
- 3 **Queries**: 由当前 query hidden state 生成，直接进入主注意力。
- 4 **Shared Key-Value Multi-Query Attention**: 接收拼接后的压缩历史与局部窗口 KV。

HCA 的数据流可以概括为：

高度压缩的历史 KV + 近期 Sliding Window KV
↓
Shared Key-Value Multi-Query Attention

与 CSA 相比，本图没有出现 Lightning Indexer、Index Scores、Top-k Selector 或 Selected Compressed KV Entries。这只能证明架构页没有展示第二段显式索引路径，不能据此断言底层不存在掩码、布局处理或其他内核优化。

两类注意力的差异如下：

对比维度	CSA / C4A	HCA / C128A
历史压缩率	约 4:1	约 128:1
显式 Indexer	展示 Lightning Indexer 与 Top-k	架构图未展示 CSA 式路径
历史参与方式	从压缩历史中选择 Top-k	压缩历史与局部 KV 直接拼接
近期 token	由 SWA 覆盖	同样由 SWA 覆盖

对比维度	CSA / C4A	HCA / C128A
主要取舍	条目更多，需要打分和选择	条目更少、路径直接，但历史压缩更强

若恰好积累了 128 个可压缩 token，按比例将对应一个历史条目 C_0 。后续尚未形成新压缩条目的近期 token 继续保存在 SWA 中：

$$[C_0] + [\text{近期 SWA KV}] \rightarrow \text{主注意力}$$

继续积累可压缩历史后，历史部分可演进为 $[C_0, C_1]$ ，再与当时的局部窗口拼接。这里表达的是数量关系，不代表材料已经确认具体发射时刻、warm-up 或边界行为。

还要注意，压缩比例中的 128 不能与后文用于调度的 `block_size=256` 混淆。前者描述原生 token 与压缩条目的数量关系，后者描述系统进行调度和缓存匹配时采用的逻辑粒度。

演讲者把 CSA、HCA 与 SWA 的组合视为支持约一百万 token 上下文并控制 KV Cache 规模的关键设计。^[^T03T07] 但材料没有任务质量对照、精度消融或分层贡献实验，因而无法判断 128:1 压缩对不同任务的影响，也不能把长上下文能力归因于 HCA 单一机制。

HCA 以更强的历史压缩换取更直接的架构路径，但架构图还不能回答它究竟节省多少显存。下一步必须把压缩比例换算为逐块、逐 token 和逐序列的存储账目。

四、从 256-token 逻辑块复核 9.62 GiB

压缩率本身不是显存数字。计算一个 1M-token 请求的缓存容量，至少还需要知道：

- 调度器按多少原生 token 划分逻辑块。
- 每个逻辑块保存多少压缩条目。
- 每条记录的表示维度或实际存储字节数。
- 模型包含多少 C4A 和 C128A 层。
- 缓存使用 FP16、FP8 还是其他格式。

为什么看下面这张容量页？因为它同时给出了逻辑块、三类随上下文增长的缓存和层数统计，可以从底层字节数复核 9.62 GiB/sequence。



KV Cache Management: Overview

C4A (CSA): compressed sparse attention
C128A (HCA): highly compressed attention

The hidden (yet simple) math behind the numbers:

- Assuming `block_size=256`
 - C4: 64 entries per block
 - C128: 2 entries per block
 - Referred to as `storage_block_size` in code
- C4 indexer (`head_dim=128`):
 - FP16: $64 \times 128 \times 2B = 16384 \text{ B/block}$, or 64 B/token
 - FP8: $64 \times 132B = 8448 \text{ B/block}$, or 33 B/token
 - FP4: $64 \times 68B \dots$
- C4 attention (`head_dim=512`):
 - FP16: $64 \times 512 \times 2B = 65536 \text{ B/block}$, or 256 B/token
 - FP8: $64 \times 584B = 37376 \text{ B/block}$, or 146 B/token
- C128 attention (`head_dim=512`):
 - FP16: $2 \times 512 \times 2B = 2048 \text{ B/block}$, or 8 B/token
 - FP8: $2 \times 584B = 1168 \text{ B/block}$, or ~4.6 B/token

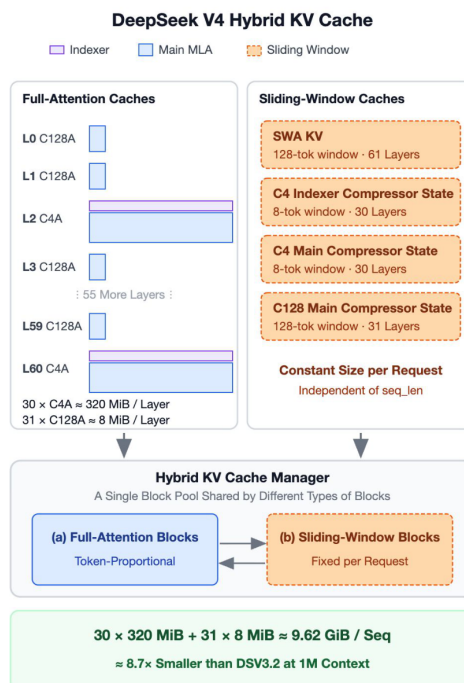


图 4：逻辑块、缓存分类与逐块容量计算。来源：演讲 PPT，第 5 页。

先区分两个概念：

- 逻辑 `block_size`**：按原生 token 定义，这里固定为 256。
- `storage_block_size`**：一个物理缓存块实际保存的压缩条目数。C4 为 64，C128 为 2。

因此，256 个原生 token 在 C4 中对应 64 个压缩条目，在 C128 中对应 2 个。压缩改变实际存储数量，但调度、KV Cache block 划分和 prefix-cache matching 仍使用统一的 256-token 逻辑粒度。

图中随上下文长度增长的缓存有三类：

- C4 Indexer Cache。
- C4 Attention Cache。
- C128 Attention Cache。

右侧还展示了 SWA 与 compressor state。它们受窗口上限约束，达到上限后主要表现为每请求近似固定的开销。页面中的 $30 \times 320 \text{ MiB} + 31 \times 8 \text{ MiB}$ 只展开前三类随 token 增长的缓存，没有列出窗口状态的完整总量。

FP16：从块容量算到逐 token 开销

设：

- N 为逻辑块中的原生 token 数， $N=256$ ；
- S 为 `storage_block_size`；
- D 为 FP16 表示下每个条目的维度；

- W 为元素宽度，FP16 中 W=2 B。

则一个物理块的容量为：

$$B_block=S \times D \times W$$

平均到每个原生 token：

$$B_token=(B_block) / (N)$$

代入页面参数：

缓存类别	每块条目数	FP16 块容量	FP16 每原生 token	FP8 页面口径
C4 Indexer	64	$64 \times 128 \times 2=16,384 \text{ B}$	64 B	$64 \times 132=8,448 \text{ B/block}$ ，即 33 B/token
C4 Attention	64	$64 \times 512 \times 2=65,536 \text{ B}$	256 B	$64 \times 584=37,376 \text{ B/block}$ ，即 146 B/token
C128 Attention	2	$2 \times 512 \times 2=2,048 \text{ B}$	8 B	$2 \times 584=1,168 \text{ B/block}$ ，即约 4.6 B/token

以 C4 Indexer 为例：

- 256 个原生 token 对应 64 个 Indexer 条目。
- 每条包含 128 个 FP16 元素。
- 一个块占用 $64 \times 128 \times 2=16,384 \text{ B}$ 。
- 除以 256，得到 64 B/native-token。

一个 C4A 层还需要主 Attention Cache，因此逐 token 开销为：

$$64+256=320 \text{ B/native-token}$$

C128A 对应：

$$8 \text{ B/native-token}$$

C4A 在容量表中包含两部分，是因为 Indexer 和主注意力各自需要缓存；C128A 的表项则只有 C128 Attention。

9.62 GiB 在什么条件下成立

页面统计对应 30 个 C4A 层和 31 个 C128A 层。演讲材料还称，示例模型共有 61 层，前两层为 C128A，后续 C4A 与 C128A 交错。PPT 折叠了中间层，因此不能只凭图面恢复完整的逐层顺序。

若把 “1M context” 按 $2^{20}=1,048,576$ 个 token 计算，每个 C4A 层需要：

$$320 \text{ B/token} \times 2^{20} \text{ token} =320 \text{ MiB}$$

每个 C128A 层需要：

$$8 \text{ B/token} \times 2^{20} \text{ token} =8 \text{ MiB}$$

总量为：

$30 \times 320 + 31 \times 8 = 9,848 \text{ MiB}$
$9,848 \div 1,024 = 9.6171875 \text{ GiB} \approx 9.62 \text{ GiB/sequence}$

因此，9.62 GiB/sequence 只适用于以下条件：

- 上下文按 2^{20} 个原生 token 计算。
- 三类随 token 增长的缓存采用 FP16。
- 层配置为 30 个 C4A 层和 31 个 C128A 层。
- 统计对象是 C4 Indexer、C4 Attention 和 C128 Attention Cache。

如果把 1M 理解为十进制的 1,000,000 token，同一公式得到 9.848 GB，约为 9.17 GiB，而不是 9.62 GiB。页面同时采用 320 MiB/■ 与 9.62 GiB，对应二进制换算口径。

FP8 为什么不能按维度直接减半

FP8 行中的 132 B/entry 和 584 B/entry 是页面给出的实际存储量，不是新的 head_dim。因此，不能把 FP16 的 128 或 512 简单减半，也不能把 132、584 当成元素数量。

按页面口径：

- C4A 每原生 token 为 33+146=179 B。
- C128A 每原生 token 约为 4.6 B。

演讲者进一步估计，同一 1M-token sequence 使用 FP8 KV Cache 后，可比 FP16 再节省约 50%，每序列显存降至约 5 GB。这个数字只有口述来源，缺少完整配置与实测条件，只能视为估计。[^T08]

页面还标注，上述 1M-context 缓存相对 DSV3.2 约小 8.7 倍，但没有展开对照侧的精度、缓存组成和计算过程。因此，8.7 倍只能保留在本页的特定场景中，不能泛化为模型整体的固定倍率。

FP4 行只展示 C4 Indexer 的 64×68B，其余信息不足，不能继续补算整层或整序列容量。

这笔账说明，压缩率必须和逻辑块、物理条目、表示格式及层数一起使用，才能得到可复核的结果。它也暴露了下一层问题：不同缓存的尺寸与增长规律差异很大，却仍要共享同一套调度和内存管理机制。

五、Hybrid KV Cache Manager：统一调度，异构装箱

调度器和前缀缓存希望请求按统一粒度推进，底层缓存却包含两种容量模式：

- Full Attention 类缓存随上下文 token 数增长。
- SWA 与 compressor state 受窗口限制，达到上限后主要表现为每请求近似固定。

如果强迫所有对象采用相同物理块，显存利用率会下降；如果为每类缓存建立独立的调度语义，prefix caching 和请求管理又会变得复杂。

vLLM 的处理方式是保留统一的 256-token 逻辑块，同时允许各缓存组使用不同的物理 block size。对于受窗口限制的状态，还需要分别记录其窗口范围。

为什么看下面这张布局图？重点不是从模糊小格中恢复完整公式，而是理解异构缓存如何进入三个共享 tensor，以及蓝色 **Wasted** 区域为什么产生。



KV Cache Management: Flexible Memory Sharing

- Uniform logical block size
 - Defined on native tokens, fixed to be 256
 - Basic unit for scheduling and cache hit
- Compressor state as SWA
 - C4: window_size=8
 - C128: window_size=128

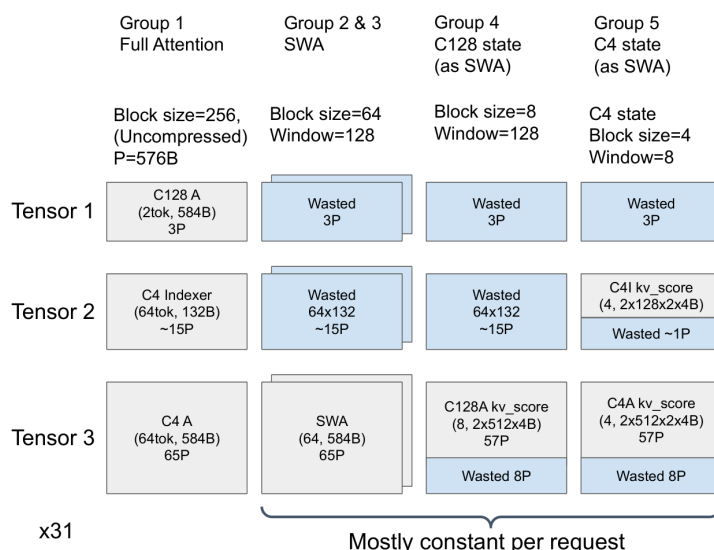


图 5：统一逻辑块下的异构缓存装箱关系。来源：演讲 PPT，第 6 页。

三个容易混淆的尺度

概念	含义	本页作用
逻辑 <code>block_size</code>	调度器一次处理的原生 token 范围	固定为 256，也是 prefix-cache matching 粒度
物理 block size	某个缓存组一次实际存放的条目数	随缓存类型变化
<code>window_size</code>	特定 SWA 或 compressor state 需要覆盖的历史范围	决定窗口受限状态的生命周期

页面能够确认的参数如下：

缓存组	物理 block size	已确认的 <code>window_size</code>	容量特征
Full Attention	256	不受本页窗口限制	随上下文增长
SWA	64	材料未确认	窗口填满后，每请求近似固定
C128 compressor state	8	128	按 SWA 方式管理

缓存组	物理 block size	已确认的 window_size	容量特征
C4 compressor state	4	8	按 SWA 方式管理

这里的 128 是 **C128 compressor state** 的窗口大小，不是已经证实的通用 SWA 窗口长度。材料只确认普通 SWA 组的物理 block size 为 64；Sliding Window KV 的精确窗口长度仍需通过配置或源码核验。

C128 state 和 C4 state 不是普通 KV 条目，但它们都有窗口上限，因此被抽象为 SWA 类缓存，以复用已有的分配与缓存管理机制。

“每请求近似固定”也有前提：窗口已经填满。在此之前，占用仍会随请求推进而增长；不同缓存组的固定上限也不相同。

三个 tensor 与内部碎片

图中把异构缓存放入三个物理 tensor。页面与口述较明确的 Full Attention 载荷包括：

- C128A 在 Tensor 1 中约占 3P。
- C4 Indexer 在 Tensor 2 中约占 15P。
- C4 Attention 在 Tensor 3 中约占 65P。

演讲者说明 1P=576B，并将该对齐粒度归因于 kernel 访存要求。[^T11] 页面部分小字不清晰，因此不应根据方框位置恢复完整布局公式。

右侧矩阵中的列表示不同缓存组，行表示三个 tensor：

- 1 随上下文增长的 Group 1 同时包含 C128A、C4 Indexer 和 C4 Attention，因此三个 tensor 都有有效载荷。
- 2 SWA 使用自己的物理块大小，映射到相应 tensor 槽位。
- 3 C128 compressor state 以 8 个条目为物理块，状态窗口为 128。
- 4 C4 compressor state 以 4 个条目为物理块，状态窗口为 8。
- 5 某个缓存组无法利用复合布局中的全部槽位时，剩余空间成为 Wasted 区域。

蓝色区域不是有效模型状态，而是统一 tensor 形状、联合分配和对齐要求产生的 padding，也就是内部碎片。

材料没有给出全模型碎片率，也没有展示不同 batch 和请求组合下的实测浪费，不能把页面中的局部空洞直接换算为整体显存效率。

长请求与大量短请求为什么需要共享池

考虑两种负载。

第一种只有一个长请求。上下文从 256 token 增长到 1024 token 时，逻辑上从一个块扩展到四个。Full Attention 类缓存持续申请新块，SWA 与 compressor state 则在各自窗口填满后不再等比例增长。此时主要压力来自随 token 数增长的容量。

第二种由大量短请求组成。每个请求的上下文可能不长，却都需要自己的窗口缓存与 compressor state。随着活跃请求数增加，每请求状态可能先于长上下文缓存形成压力。

设第 i 个请求长度为 L_i ，活跃请求数为 R ，容量趋势可以粗略表示为：

$$M \approx a \sum_i L_i + bR$$

其中：

- a 表示随 token 增长的缓存成本。
- b 表示窗口与 compressor state 带来的每请求成本。

一个超长请求主要放大第一项，大量短请求主要放大第二项。该式只描述增长关系，不是精确显存模型；物理块大小、窗口填充程度、prefix-cache 命中和对齐碎片都会影响真实结果。

Hybrid KV Cache Manager 让两类状态共享同一 block pool，并可按工作负载调整容量占比。它解决的是“剩余显存更多服务长上下文，还是更多服务高并发请求”的装箱问题，但不能仅凭总显存推导任意负载下的最大并发数。

复用能力与命中粒度

把 compressor state 作为 SWA 管理后，演讲者称系统可以复用现有 KV Cache Manager 的内存分配和 prefix caching，并让 Prefill/Decode 分离时的缓存传输及后续 offloading 支持更直接。[^T05T13]

其中，PPT 明确展示的是“Compressor state as SWA”；prefix caching、PD 分离与 offloading 的复用收益主要来自口述，不能扩大为已经完整验证的功能清单。

当前 prefix-cache matching 仍采用 256-token 粒度。演讲中还讨论了一种 checkpoint 式折中：例如每 1024 token 保存一次窗口状态，以减少状态保存次数；如果采用这种假设方案，命中粒度也只能是 1024 token。这里的 1024 是讨论示例，不是当前默认配置。[^T13T14]

缓存“放在哪里”的问题至此已有答案：调度继续使用统一的 256-token 逻辑块，底层则以 256、64、8、4 等物理块适配不同状态。接下来要看这些数据如何在 C4A Decode 中被并行生产、选择和消费。

六、C4A Decode：两条流在 Flash MLA 前汇合

CSA 把主注意力准备和历史选择拆成两项近似独立的工作。执行层的关键问题是：怎样让两条路径尽可能重叠，同时避免一系列小算子把时间耗在 HBM 往返和 kernel launch 上？

这里需要两个概念：

- **CUDA stream** 允许不存在直接数据依赖的 GPU 工作并发推进。
- 对规模较小的逐元素算子，性能经常受 HBM 数据搬运和启动开销限制，而不是算术吞吐限制。

为什么看下面这张执行图？因为它明确限定在 **C4A Decode path**，并标出了默认流、Indexer 流、三类 Top-K 输出和 Flash MLA 前的汇合关系。

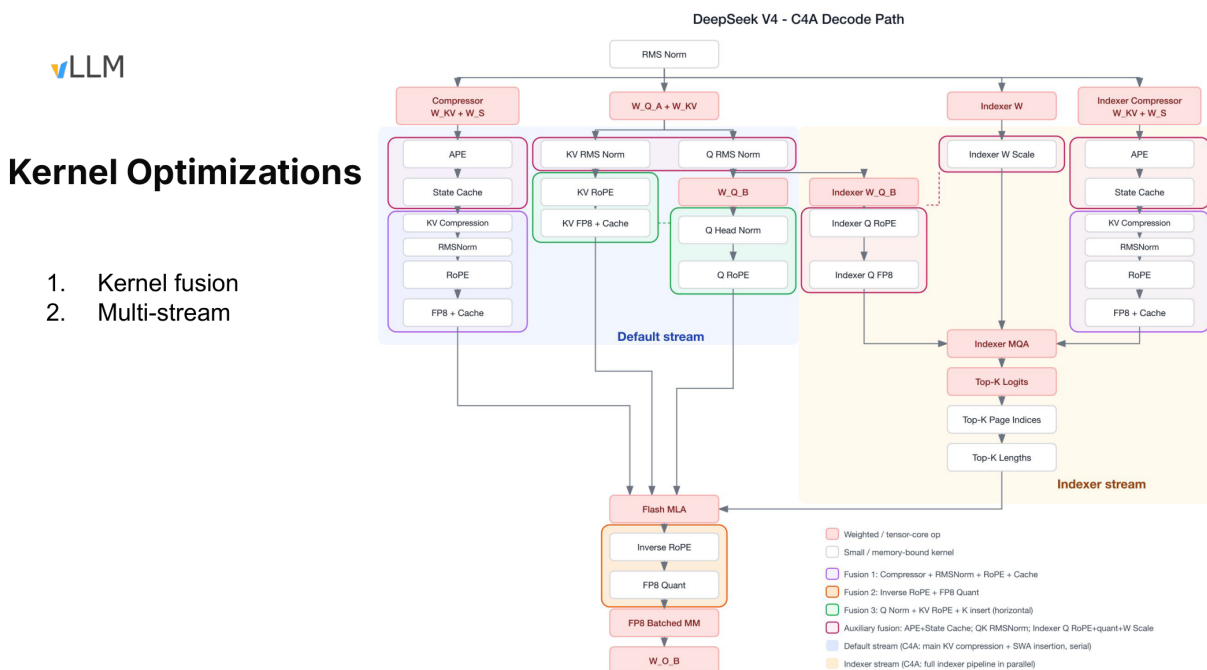


图 6：C4A Decode 的 kernel fusion 与 multi-stream 执行图。来源：演讲 PPT，第 7 页。

默认流与 Indexer 流

蓝色区域表示 **Default stream (默认流)**，负责主注意力输入、compressor 和缓存相关准备。

黄色区域表示 **Indexer stream (索引器流)**，运行 Lightning Indexer，为压缩历史条目打分并生成选择信息。

阶段	默认流	Indexer 流	跨流关系
输入准备	主 Q/KV、compressor 与缓存输入	Indexer W、Indexer Q、Indexer compressor	可分别推进
流内计算	归一化、RoPE、Cache 写入等	Indexer MQA 与候选评分	各自保持依赖
选择输出	等待稀疏访问所需信息	生成 Top-K logits、page indices、lengths	开始形成汇合条件
稀疏注意力	提供主注意力输入	提供 Indexer 选择结果	Flash MLA 等待两侧
后处理	Inverse RoPE、量化及输出计算	本步索引任务结束	回到主路径

Indexer 产生三类结果：

- Top-K logits**：选中条目的评分信息。
- Top-K page indices**：指出需要读取的缓存页面。
- Top-K lengths**：执行图列出的长度信息；其具体含义和编码方式需要结合对应版本源码确认。

Flash MLA 同时依赖默认流准备的主注意力输入和 Indexer 流产生的选择结果。因此，它是本次 Decode step 的关键同步点：任意一侧尚未完成，Flash MLA 都不能开始。

双流背景只表示存在并行机会，不代表所有节点都能完全重叠。实际效果仍受输入形状、GPU 资源竞争和运行时调度影响。

一个 Decode step 的事件序列

单步 Decode 可以抽象成五个事件：

- 1 hidden state 到达后，默认流开始准备主注意力的 Q/KV，并推进 compressor 与缓存操作。
- 2 Indexer 流同时准备 Indexer W、Indexer Q 和自身的 compressor 输入。
- 3 Indexer MQA 汇合流内输入，计算候选评分，随后产生 Top-K logits、页面索引和长度信息。
- 4 两条流在 Flash MLA 前同步；先完成的一侧必须等待另一侧。
- 5 Flash MLA 读取选中的压缩 KV 与局部窗口 KV，输出再进入 Inverse RoPE、FP8 Quant 等后处理。

若默认流准备时间为 T_m ，Indexer 流为 T_i ，忽略资源争用和同步开销，并行准备阶段的理想下界接近：

$$T_{prepare} \approx \max(T_m, T_i)$$

串行执行则接近 $T_m + T_i$ 。

例如，两侧分别需要 4 和 3 个抽象时间单位，理想并行准备时间接近 4，而不是 7。这个例子只解释依赖关系，不是演讲中的性能测量；真实延迟还要计入跨流同步和 Flash MLA 本身。

三类融合分别减少什么

Multi-stream 解决“本可并行却被串行化”的问题，kernel fusion 则处理“操作过于零碎”的问题。

第一类是顺序链融合：

Compressor + RMSNorm + RoPE + Cache

这些操作沿同一数据链执行。如果拆成多个 kernel，中间结果需要反复写入和读取 HBM。融合可以减少中间张量往返，同时降低启动次数。

第二类也是顺序融合：

Inverse RoPE + FP8 Quant

Flash MLA 的输出先经过逆旋转，再进行量化。把两步连接起来，可以避免 Inverse RoPE 的结果作为独立中间张量落入 HBM。

第三类是横向融合：

Q Norm + KV RoPE + K Insert

这些小操作不一定存在严格的前后依赖，但单独启动时利用率可能不足。横向融合让不同工作共享一次 kernel launch，从而降低零碎任务的调度成本。

页面还展示了 APE 与 State Cache、Q/K RMSNorm、Indexer Q 的 RoPE 与量化、W Scale 等辅助融合。它们遵循两个共同原则：

- 对连续数据链，减少中间值写回。
- 对规模较小的独立操作，减少 kernel 数量和启动成本。

这不意味着所有大型计算都适合无条件融合。材料中的优化解释主要针对受 HBM 搬运与 kernel launch 限制的小算子。

Inverse RoPE 的解释边界

RoPE (Rotary Position Embedding, 旋转位置编码) 通过旋转表示注入位置信息。

演讲转写给出的解释是：共享 K/V 路径在计算前对共同表示施加 RoPE，影响会随之传到输出中的 V；Decode 路径随后使用 **Inverse RoPE** 抵消这部分旋转，使数学效果接近只对 K 应用 RoPE。[^T16]

PPT 能够确认执行图中存在 Inverse RoPE 节点，但没有给出完整公式和训练侧推导。因此，上述因果关系应视为演讲者解释，具体数学实现仍需源码核验。

性能数字不能脱离条件

演讲者称，对于受 HBM 搬运和启动开销限制的小算子，融合可能带来约 2~4 倍收益，个别情况下可能更高。材料没有提供硬件、输入 shape、比较基线，也没有拆分各项 fusion 的延迟或吞吐，因此这只能作为特定条件下的经验观察。[^T16]

MegaMoE 延续类似思路：把专家计算、通信以及部分前后小算子融合到较大的内核。演讲者将整条 MoE 路径描述为可压缩到“约一次 kernel launch”，目标是减少 CPU 启动开销并简化调用接口；这同样不是附带完整实验条件的吞吐结论。[^T03T18]

本页只覆盖 C4A Decode。演讲者称 Prefill 因不读取此前的 KV Cache 而相对简单，但材料没有相应的完整执行图；C128A 也不属于本图描述的路径。[^T15] 因而这里的双流组织、融合组合和收益范围都不能直接外推。

七、从架构图落到代码：核验入口与证据边界

前面的分析建立了模型结构、缓存对象和执行路径之间的关系，但架构解释不等于源码事实。

为什么看最后这张图？它提供了继续验证的入口，却没有展示完整调用图。正确用法是从待核验问题进入对应文件，而不是根据文件名猜测函数行为。

Key Source Code Files

- Entry + model definition: `models/deepseek_v4.py`
- High-level attention layer: `layers/deepseek_v4_attention.py`
 - Including the indexer and the main attention module
- Compressor (for both CSA and HCA): `layers/deepseek_compressor.py`
 - Including the compressor state cache and metadata builder
- SWA KV cache: `mha/sparse_mha.py`
 - Including the SWA cache and metadata builder
- MegaMoE: inlined in `models/deepseek_v4.py`
- KV cache layout: `core/kv_cache_utils.py::_get_kv_cache_config_deepseek_v4()`

图 7：模型、注意力、压缩器、SWA 缓存、MegaMoE 与 KV Cache 布局的源码导航。来源：演讲 PPT，第 8 页。

按问题整理后，源码导航如下：

核验问题	建议入口	材料明确的信息	仍需源码确认
模型怎样组装子模块	<code>models/deepseek_v4.py</code>	模型定义位于此处	初始化顺序、配置映射和前向路径
Indexer 如何衔接主注意力	<code>layers/deepseek_v4_attention.py</code>	包含 Indexer 与主 Attention Module	输出传递、同步及边界处理
CSA/HCA 如何维护压缩状态	<code>layers/deepseek_compressor.py</code>	包含 compressor、状态缓存和 metadata builder	状态创建、更新、复用与重计算
SWA 缓存如何参与执行	<code>mha/sparse_mha.py</code>	包含 SWA Cache 和 metadata builder	SWA 与压缩历史的同步规则
MegaMoE 在哪里接入	<code>models/deepseek_v4.py</code>	MegaMoE 内联于模型文件	对应函数及实际融合边界
特殊缓存布局如何生成	<code>core/kv_cache_utils.py::_get_kv_cache_config_deepseek_v4()</code>	给出布局配置入口	逻辑块、物理存储与分配结果的映射

一种可操作的阅读顺序是：

- 1 从 `_get_kv_cache_config_deepseek_v4()` 查看布局配置。
- 2 追踪配置如何进入 KV Cache Manager。
- 3 查看 `deepseek_compressor.py` 如何描述压缩状态和 metadata。
- 4 查看 `sparse_mha.py` 如何描述 SWA Cache。
- 5 回到 `deepseek_v4_attention.py`，确认两类状态如何进入注意力。

6 最后检查 `deepseek_v4.py` 如何组装模型与 MegaMoE。

这只是核验顺序，不代表材料已经证明运行时调用关系。

例如，要验证“压缩条目与近期 SWA token 是否完整覆盖上下文”，可以进行一次最小状态追踪：

- 1 记录一个请求从布局入口获得的缓存配置。
- 2 对照 compressor metadata，确认已压缩状态覆盖哪些 token。
- 3 对照 SWA metadata，确认尚未形成压缩条目的近期 token 如何保存。
- 4 回到注意力入口，确认两类状态是否在同一次执行中被正确读取。

如果压缩状态与 SWA 状态之间出现空洞或重叠，仅凭架构图无法判断它来自 padding、窗口边界、布局约束还是实现缺陷，必须结合对应版本的代码与运行状态继续验证。

版本与规划边界

这些源码路径没有附带仓库分支、版本号或提交哈希，只能代表演讲对应的代码快照。后续版本中，文件可能移动，符号也可能重命名。

封面的“2026 年 5 月”只能视为演讲时间，不能作为模型或软件的正式发布日期。

演讲还提出让模型定制 KV Cache planning 与 allocation，把 DeepSeek V4 的特殊逻辑移入模型相关代码，以减少其对通用 KV Cache Manager 和其他模型的影响。[^T12] 减少 padding、扩展 offloading 等也被列为后续方向。

这些内容属于演讲时的规划，不是已发布能力，也不构成交付承诺。

结论：百万 token 推理是一条完整的系统链路

- 1 **DeepSeek V4 的长上下文设计不是单一稀疏注意力。** CSA 通过约 4:1 压缩和 Top-k 选择分别控制容量与计算；HCA 通过约 128:1 压缩减少长期历史条目；两类层配置都依靠 SWA 覆盖近期 token。
- 2 **容量压缩与计算稀疏解决不同问题。** Token compression 改变实际保存的历史条目数，Top-k selection 只改变当前计算访问的范围。只有前者直接降低 KV Cache 随序列增长的容量。
- 3 **压缩把同构 KV Cache 转换成异构状态管理问题。** C4/C128 主缓存、Indexer Cache、SWA 和 compressor state 具有不同尺寸、窗口与增长规律，难以用单一物理块高效承载。
- 4 **256-token 逻辑块维持统一调度语义。** `storage_block_size` 和不同物理 block size 负责映射真实存储；逻辑粒度、物理块大小与 `window_size` 是三个不同概念。
- 5 **Hybrid KV Cache Manager 在长上下文与高并发之间动态装箱。** 长请求主要消耗随 token 增长的缓存，大量短请求会放大每请求窗口和状态开销；共享 pool 可以调节容量比例，但统一布局也会引入 padding 与内部碎片。
- 6 **C4A Decode 的关键同步点是 Flash MLA。** 默认流准备主注意力输入，Indexer 流产生 Top-K logits、page indices 和 lengths；两侧都完成后，稀疏注意力才能执行。`lengths` 的具体字段语义仍需源码确认。

- 7 **双流与 kernel fusion 优化的是关键路径、HBM 往返和启动开销。** 约 2~4 倍收益及 MegaMoE “约一次 kernel launch” 都缺少完整实验条件，只能保留为演讲者的条件性观察。
- 8 **9.62 GiB 不是脱离配置的模型常数。** 它依赖 2^{20} 个 token、FP16、30 个 C4A 层、31 个 C128A 层，以及页面统计的三类随 token 增长缓存；约 5 GB 和 8.7 倍同样必须保留各自的场景与证据限制。

明确局限

现有材料不足以回答：

- CSA 与 HCA 分别造成多少精度或任务质量变化。
- Top-k、通用 SWA 窗口长度及 compressor 的完整模型配置。
- 首个压缩条目的发射时刻、warm-up、padding 与边界对齐规则。
- 两条 compressor 路径是否共享参数及其具体算法。
- 单个压缩条目的严格语义覆盖区间。
- Top-K `lengths` 的具体编码对象与字段语义。
- 不同 batch、请求长度和缓存命中率下的真实碎片率。
- 双流和各项 fusion 在特定硬件与 shape 下的独立收益。
- 演讲源码路径对应的准确提交版本。
- 模型定制缓存规划、进一步减少 padding 和 offloading 的实际落地状态。

现有证据能够支持的是一条完整的工程因果链：压缩注意力减少历史条目，却产生异构缓存；统一逻辑块维持调度与命中语义，差异化物理块承载真实状态；双流执行和算子融合再把这些状态接入可运行的 Decode 关键路径。

百万 token 推理由此成为模型表示、缓存规划、内存布局与 GPU 执行共同约束的系统工程，而不是某个注意力算子的孤立结果。

[^T02T03]: 口述来源：转写 T02、T03。mHC 的结构描述及“对后续接口没有影响”的限定来自演讲者说明。

[^T03]: 口述来源：转写 T03。Hash Routing 的前三层配置及“可能更易训练”均为演讲者转述。

[^T04]: 口述来源：转写 T04。“最多 512 个”未获 PPT 参数表确认。

[^T06]: 口述来源：转写 T06。容量压缩与计算稀疏的职责区分来自演讲者回答。

[^T03T07]: 口述来源：转写 T03、T07。约一百万 token 的因果归属缺少质量消融实验。

[^T08]: 口述来源：转写 T08。FP8 “再节省约 50%” “约 5 GB” 为估计。

[^T11]: 口述来源：转写 T11。`1P=576B` 及其与 kernel 访存要求的关系来自演讲者解释。

[^T05T13]: 口述来源：转写 T05、T13。Prefix caching、PD 分离、缓存传输和 offloading 的复用收益未由页面完整展开。

[^T13T14]: 口述来源：转写 T13、T14。1024-token checkpoint 是假设示例；当前命中粒度仍为 256 token。

[^T16]: 口述来源：转写 T16。Inverse RoPE 的数学解释及约 2~4 倍融合收益均缺少完整公式或实验条件。

[^T03T18]: 口述来源：转写 T03、T18。MegaMoE “约一次 kernel launch” 是实现目标描述，不是完整性能测试。

[^T15]: 口述来源：转写 T15。Prefill 相对简单的说明没有相应的完整执行图。

[^T12]: 口述来源：转写 T12。模型定制 KV Cache planning 与 allocation 是演讲时的后续规划。